Chapter 2: Overview of the Data Mining Process

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

We use the pandas, the Python data anlysis library, for handling data.

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

Make sure MLBA package is available. It includes data sets and utility functions for 'Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python' (2nd edition).

In [2]:
```
pip install mlba
```

```
Requirement already satisfied: mlba in /opt/anaconda3/lib/python3.13/site-pa
ckages (2.0.2)
Note: you may need to restart the kernel to use updated packages.
```

In [85]:
```
%matplotlib inline
import mlba
import numpy as np # A powerful library for numerical computations and array
import pandas as pd # A popular library for data manipulation and analysis u
import matplotlib.pyplot as plt  # A widely used module for creating static,
from sklearn.model_selection import train_test_split # Import a function to
from sklearn.metrics import r2_score # Import a function to compute the R² s
from sklearn.linear_model import LinearRegression # Import the Linear Regres
```

Note that we import some of the packages using an alias notation, to make the code more readable.

```
import <package> as <alias>
```

The aliases `np`, `pd`, and `plt` are commonly used in the data science community.

Loading and Looking at the Data in Python

Table 2.3

Load the West Roxbury data set. In the example shown in Table 2.1 (home values in West Roxbury, Boston, in 2014), 14 variables are recorded for over 5000 homes. The spreadsheet is organized in such a way that each row represents a home - the first home's assessed value was 344,200 dollars, its tax was 4,330 dollars, its size was 9,965 SQ.FT, it was built in 1880, and so on.

In [7]:
```
# VB67
# loading the given dataset.
housing_df = mlba.load_data('WestRoxbury.csv')
```

Determine the shape of the data frame. It has 5802 rows and 14 columns

In [9]:
```
# VB67
# This code cell gives us the shape of the dataset.
housing_df.shape
```

Out[9]:
```
(5802, 14)
```

Show the first five rows of the data frame

In [10]:
```
# VB67
# This code cell gives us the 5 rows of the dataset.
housing_df.head()
```

Out[10]:

	TOTAL VALUE	TAX	LOT SQFT	YR BUILT	GROSS AREA	LIVING AREA	FLOORS	ROOMS	BEDROOMS	FULL BATH
0	344.2	4330	9965	1880	2436	1352	2.0	6	3	1
1	412.6	5190	6590	1945	3108	1976	2.0	10	4	2
2	330.1	4152	7500	1890	2294	1371	2.0	8	4	1
3	498.6	6272	13773	1957	5032	2608	1.0	9	5	1
4	331.5	4170	5000	1910	2370	1438	2.0	7	3	2

Cleanup

Preprocessing and cleaning up data is an important aspect of data analysis.

Show the column names.

In [12]:
```
#VB67
#printing all column names
housing_df.columns
```

Out[12]:
```
Index(['TOTAL VALUE ', 'TAX', 'LOT SQFT ', 'YR BUILT', 'GROSS AREA ',
       'LIVING AREA', 'FLOORS ', 'ROOMS', 'BEDROOMS ', 'FULL BATH',
       'HALF BATH', 'KITCHEN', 'FIREPLACE', 'REMODEL'],
      dtype='object')
```

Note that some column titles end with spaces and some consist of two space separated words. For further analysis it's more convenient to have column names which are single words.

In [15]:
```
#VB67
#renaming the columns for consistent naming by replacing space with underscc
housing_df = housing_df.rename(columns={'TOTAL VALUE ': 'TOTAL_VALUE'})
housing_df.columns
```

Out[15]:
```
Index(['TOTAL_VALUE', 'TAX', 'LOT SQFT ', 'YR BUILT', 'GROSS AREA ',
       'LIVING AREA', 'FLOORS ', 'ROOMS', 'BEDROOMS ', 'FULL BATH',
       'HALF BATH', 'KITCHEN', 'FIREPLACE', 'REMODEL'],
      dtype='object')
```

We therefore strip trailing spaces and replace the remaining spaces with an underscore _. This changes only one column — the one that exactly matches 'TOTAL VALUE ' (with a trailing space) — into 'TOTAL_VALUE'.

In [18]:
```
#VB67
#renaming the remaining columns for consistent naming by replacing space wit
housing_df.columns = [s.strip().replace(' ', '_') for s in housing_df.colum
housing_df.columns
```

Out[18]:
```
Index(['TOTAL_VALUE', 'TAX', 'LOT_SQFT', 'YR_BUILT', 'GROSS_AREA',
       'LIVING_AREA', 'FLOORS', 'ROOMS', 'BEDROOMS', 'FULL_BATH', 'HALF_BAT
H',
       'KITCHEN', 'FIREPLACE', 'REMODEL'],
      dtype='object')
```

Instead of using the `rename` method, we create a modified copy of `columns` and assign to the `columns` field of the data frame. This changes all column names.

Accessing subsets of the data

Pandas uses two methods to access rows in a data frame; `loc` and `iloc`. The `loc` method is more general and allows accessing rows using labels. The `iloc` method on the other hand only allows using integer numbers. To specify a range of rows use the slice notation, e.g. `0:9`.

> Note that in contrast to R, Python uses 0-indexing, which means that indices start at 0 and not at 1.

To show the first four rows of the data frame, you can use the following commands.

In [26]:
```
#VB67
housing_df.loc[0:3]
# for loc, the second index in the slice is inclusive
# loc[a:b] gives rows a to b, inclusive
```

Out[26]:

	TOTAL_VALUE	TAX	LOT_SQFT	YR_BUILT	GROSS_AREA	LIVING_AREA	FLOORS
0	344.2	4330	9965	1880	2436	1352	2.0
1	412.6	5190	6590	1945	3108	1976	2.0
2	330.1	4152	7500	1890	2294	1371	2.0
3	498.6	6272	13773	1957	5032	2608	1.0

In [25]:
```
#VB67
housing_df.iloc[0:4]
# for iloc, the second index in the slice is exclusive
# iloc[a:b] gives rows a to b-1
```

Out[25]:

	TOTAL_VALUE	TAX	LOT_SQFT	YR_BUILT	GROSS_AREA	LIVING_AREA	FLOORS
0	344.2	4330	9965	1880	2436	1352	2.0
1	412.6	5190	6590	1945	3108	1976	2.0
2	330.1	4152	7500	1890	2294	1371	2.0
3	498.6	6272	13773	1957	5032	2608	1.0

Note the difference in the two methods with respect to the slice notation! For consistency with how slices are defined in Python, we will use the `iloc` method mostly from here on.

Next, show the first ten rows of the first column

In [23]:
```
#VB67
housing_df['TOTAL_VALUE'].iloc[0:10]
housing_df.iloc[0:10]['TOTAL_VALUE']  # the order is not important
housing_df.iloc[0:10].TOTAL_VALUE # use dot notation if the column name has
```

Out[23]:
```
0    344.2
1    412.6
2    330.1
3    498.6
4    331.5
5    337.4
6    359.4
7    320.4
8    333.5
9    409.4
Name: TOTAL_VALUE, dtype: float64
```

Show the fifth row of the first 10 columns. The `iloc` methods allows specifying the rows and columns within one set of brackets. `dataframe.iloc[rows, columns]`

In [28]:
```
#VB67
housing_df.iloc[4][0:10]
housing_df.iloc[4, 0:10]  # this is equivalent
```

Out[28]:
```
TOTAL_VALUE    331.5
TAX             4170
LOT_SQFT        5000
YR_BUILT        1910
GROSS_AREA      2370
LIVING_AREA     1438
FLOORS           2.0
ROOMS              7
BEDROOMS           3
FULL_BATH          2
Name: 4, dtype: object
```

If you prefer to preserve the data frame format, use a slice for the rows as well.

In [74]:
```
#VB67
housing_df.iloc[4:5, 0:10]
```

Out[74]:

	TOTAL_VALUE	TAX	LOT_SQFT	YR_BUILT	GROSS_AREA	LIVING_AREA	FLOORS
4	331.5	4170	5000	1910	2370	1438	2.0

Use the `pd.concat` method if you want to combine non-consecutive columns into a new data frame. The `axis` argument specifies the dimension along which the concatenation happens, 0=rows, 1=columns.

In [31]:
```
#VB67
#CONCATENATING first, second,fifth,sixth columns for rows fifth and sixth rc
pd.concat([housing_df.iloc[4:6,0:2], housing_df.iloc[4:6,4:6]], axis=1)
```

Out[31]:

	TOTAL_VALUE	TAX	GROSS_AREA	LIVING_AREA
4	331.5	4170	2370	1438
5	337.4	4244	2124	1060

To specify a full column, use the `:` on its own.

```
housing_df.iloc[:,0:1]
```

A often more practical way is to use the column name as follows

In [33]:
```
#VB67
#printing the values of column "TOTAL_VALUE"
housing_df['TOTAL_VALUE']
```

Out[33]:
```
0       344.2
1       412.6
2       330.1
3       498.6
4       331.5
        ...
5797    404.8
5798    407.9
5799    406.5
5800    308.7
5801    447.6
Name: TOTAL_VALUE, Length: 5802, dtype: float64
```

We can subset the column using a slice

In [35]:
```
#VB67
#printing the values from zero to tenth row from column "TOTAL_VALUE"
housing_df['TOTAL_VALUE'][0:10]
```

Out[35]:
```
0    344.2
1    412.6
2    330.1
3    498.6
4    331.5
5    337.4
6    359.4
7    320.4
8    333.5
9    409.4
Name: TOTAL_VALUE, dtype: float64
```

Pandas provides a number of ways to access statistics of the columns.

In [75]:
```
#VB67
print('Number of rows ', len(housing_df['TOTAL_VALUE'])) # show length of fi
print('Mean of TOTAL_VALUE ', housing_df['TOTAL_VALUE'].mean()) # show mean
```

```
Number of rows  5802
Mean of TOTAL_VALUE  392.6857149258877
```

A data frame also has the method `describe` that prints a number of common statistics

In [76]:
```
#VB67
housing_df['TOTAL_VALUE'].describe() # for one column
```

Out[76]:
```
count    5802.000000
mean      392.685715
std        99.177414
min       105.000000
25%       325.125000
50%       375.900000
75%       438.775000
max      1217.800000
Name: TOTAL_VALUE, dtype: float64
```

In [77]:
```
#VB67
housing_df.describe() # for all columns
```

Out[77]:

	TOTAL_VALUE	TAX	LOT_SQFT	YR_BUILT	GROSS_AREA	LIVIN
count	5802.000000	5802.000000	5802.000000	5802.000000	5802.000000	5802
mean	392.685715	4939.485867	6278.083764	1936.744916	2924.842123	165
std	99.177414	1247.649118	2669.707974	35.989910	883.984726	540
min	105.000000	1320.000000	997.000000	0.000000	821.000000	504
25%	325.125000	4089.500000	4772.000000	1920.000000	2347.000000	1308
50%	375.900000	4728.000000	5683.000000	1935.000000	2700.000000	1548
75%	438.775000	5519.500000	7022.250000	1955.000000	3239.000000	1873
max	1217.800000	15319.000000	46411.000000	2011.000000	8154.000000	5289

Sampling from a Database

Table 2.4

Use the `sample` method to retrieve a random sample of observations. Here we sample 5 observations without replacement.

In [41]:
```
#VB67
#printing five random samples without replacement
housing_df.sample(5)
```

Out[41]:

	TOTAL_VALUE	TAX	LOT_SQFT	YR_BUILT	GROSS_AREA	LIVING_AREA	FLOO
882	363.1	4567	5478	1988	2601	1674	
4631	458.5	5767	5299	1940	3186	1813	
2045	357.8	4501	6330	1929	2454	1456	
4929	524.4	6596	7219	1927	3437	1920	
2520	379.1	4769	5545	1925	2857	1829	

The sample method allows to specify weights for the individual rows. We use this here to oversample houses with over 10 rooms.

In [51]:

```
#VB67
#assigning more weight to the samples which have more than ten rooms in the
weights = [0.9 if rooms > 10 else 0.01 for rooms in housing_df.ROOMS]
housing_df.sample(5, weights=weights)
```

Out[51]:

	TOTAL_VALUE	TAX	LOT_SQFT	YR_BUILT	GROSS_AREA	LIVING_AREA	FLOO
5728	418.6	5265	7567	1940	2832	1568	
2453	564.7	7103	7676	1907	3596	2360	
1261	304.4	3829	4400	1925	2215	1000	
4244	443.9	5584	17680	1955	4095	1600	
5662	344.8	4337	4507	1925	1945	1225	

Preprocessing and Cleaning the Data

Types of Variables

Table 2.5

In [45]:

```
#VB67
print(housing_df.dtypes[['TOTAL_VALUE', 'FLOORS', 'REMODEL']])
# print the data type
housing_df.columns
```

```
TOTAL_VALUE     float64
FLOORS          float64
REMODEL        category
dtype: object
```

Out[45]:

```
Index(['TOTAL_VALUE', 'TAX', 'LOT_SQFT', 'YR_BUILT', 'GROSS_AREA',
       'LIVING_AREA', 'FLOORS', 'ROOMS', 'BEDROOMS', 'FULL_BATH', 'HALF_BAT
H',
       'KITCHEN', 'FIREPLACE', 'REMODEL'],
      dtype='object')
```

REMODEL contains the values “Old,” “Recent,” and missing values “NaN.” We can convert REMODEL to a categorical variable by first replacing the missing values with “None” and then changing the type of the column to category. The resulting column has three categories.

In [46]:
```
#VB67
print(housing_df.REMODEL.dtype)
housing_df.REMODEL = housing_df.REMODEL.fillna('None').astype('category')
print(housing_df.REMODEL.cat.categories)  # Show the categories.
print(housing_df.REMODEL.dtype)  # Type is now 'category'
```

```
category
Index(['None', 'Old', 'Recent'], dtype='object')
category
```

Other columns also have types.

In [53]:
```
#VB67
print(housing_df.BEDROOMS.dtype)  # BEDROOMS is an integer variable
print(housing_df.TOTAL_VALUE.dtype)  # Total_Value is a numeric variable
```

```
int64
float64
```

Show data types for all columns.

In [55]:
```
#VB67
#printing data types of all columns from the data frame.
housing_df.dtypes
```

Out[55]:
```
TOTAL_VALUE     float64
TAX               int64
LOT_SQFT          int64
YR_BUILT          int64
GROSS_AREA        int64
LIVING_AREA       int64
FLOORS          float64
ROOMS             int64
BEDROOMS          int64
FULL_BATH         int64
HALF_BATH         int64
KITCHEN           int64
FIREPLACE         int64
REMODEL        category
dtype: object
```

Handling Categorical Variables

Table 2.6

Pandas provides a method to convert categorical varables into dummy variables. We can now call `get_dummies` with the default value of `drop_first=False`.

In [56]:
```
#VB67
# Uses '_' to separate the original column name and category label.
# Makes sure the new dummy columns are stored as integers (0 or 1) rather th
housing_df = pd.get_dummies(housing_df, prefix_sep='_', dtype=int)
housing_df.columns
```

Out[56]:
```
Index(['TOTAL_VALUE', 'TAX', 'LOT_SQFT', 'YR_BUILT', 'GROSS_AREA',
       'LIVING_AREA', 'FLOORS', 'ROOMS', 'BEDROOMS', 'FULL_BATH', 'HALF_BAT
H',
       'KITCHEN', 'FIREPLACE', 'REMODEL_None', 'REMODEL_Old',
       'REMODEL_Recent'],
      dtype='object')
```

In [58]:
```
#VB67
#printing the five rows of given columns
print(housing_df.loc[:, 'REMODEL_None':'REMODEL_Recent'].head(5))
```

```
   REMODEL_None  REMODEL_Old  REMODEL_Recent
0             1            0               0
1             0            0               1
2             1            0               0
3             1            0               0
4             1            0               0
```

Missing Values

Table 2.7

To illustrate missing data procedures, we first convert a few entries for bedrooms to NA's. Randomly introduce 10 missing values into the column. Then we impute these missing values using the median of the remaining values.

In [71]:
```
#VB67
#printing the rows with valid bedrooms values.
print('Number of rows with valid BEDROOMS values before: ',
      housing_df['BEDROOMS'].count())
missingRows = housing_df.sample(10).index # Randomly selects 10 row indices
housing_df.loc[missingRows, 'BEDROOMS'] = np.nan # Sets the 'BEDROOMS' value
print('Number of rows with valid BEDROOMS values after setting to NAN: ',
      housing_df['BEDROOMS'].count())
housing_df['BEDROOMS'].count()
```

```
Number of rows with valid BEDROOMS values before:  5802
Number of rows with valid BEDROOMS values after setting to NAN:  5792
```

Out[71]:
```
np.int64(5792)
```

In [72]:
```
#VB67
# remove rows with missing values - 1st approach
```

```
reduced_df = housing_df.dropna()
print('Number of rows after removing rows with missing values: ', len(reduce
```

```
Number of rows after removing rows with missing values:  5792
```

In [73]:
```
#VB67
# Replace the missing values using the median of the remaining values - 2nd
medianBedrooms = housing_df['BEDROOMS'].median()
housing_df.BEDROOMS = housing_df.BEDROOMS.fillna(value=medianBedrooms)
print('Number of rows with valid BEDROOMS values after filling NA values: ',
      housing_df['BEDROOMS'].count())
```

```
Number of rows with valid BEDROOMS values after filling NA values:  5802
```

Normalizing (Standardizing) and Rescaling Data

Table 2.8

In [78]:
```
#VB67
from sklearn.preprocessing import MinMaxScaler, StandardScaler
df = housing_df.copy()

# Standardizing a data frame

# pandas:
norm_df = (housing_df - housing_df.mean()) / housing_df.std()

# scikit-learn:
scaler = StandardScaler()
norm_df = pd.DataFrame(scaler.fit_transform(housing_df),
                       index=housing_df.index, columns=housing_df.columns)

# Normalizing a data frame

# pandas:
rescaled_df = (housing_df - housing_df.min()) / (housing_df.max() - housing_

# scikit-learn:
scaler = MinMaxScaler()
rescaled_df = pd.DataFrame(scaler.fit_transform(housing_df),
                       index=housing_df.index, columns=housing_df.columns)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/ops/array_ops.p
y:218, in _na_arithmetic_op(left, right, op, is_cmp)
    217 try:
--> 218     result = func(left, right)
    219 except TypeError:

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/computation/exp
ressions.py:242, in evaluate(op, a, b, use_numexpr)
    240     if use_numexpr:
    241         # error: "None" not callable
--> 242         return _evaluate(op, op_str, a, b)  # type: ignore[misc]
    243 return _evaluate_standard(op, op_str, a, b)

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/computation/exp
ressions.py:131, in _evaluate_numexpr(op, op_str, a, b)
    130 if result is None:
--> 131     result = _evaluate_standard(op, op_str, a, b)
    133 return result

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/computation/exp
ressions.py:73, in _evaluate_standard(op, op_str, a, b)
     72     _store_test_result(False)
---> 73 return op(a, b)

TypeError: numpy boolean subtract, the `-` operator, is not supported, use t
he bitwise_xor, the `^` operator, or the logical_xor function instead.

During handling of the above exception, another exception occurred:

TypeError                                 Traceback (most recent call last)
Cell In[78], line 18
     12 norm_df = pd.DataFrame(scaler.fit_transform(housing_df),
     13                        index=housing_df.index, columns=housing_df.co
lumns)
     15 # Normalizing a data frame
     16
     17 # pandas:
---> 18 rescaled_df = (housing_df - housing_df.min()) / (housing_df.max() -
housing_df.min())
     20 # scikit-learn:
     21 scaler = MinMaxScaler()

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/ops/common.py:7
6, in _unpack_zerodim_and_defer.<locals>.new_method(self, other)
     72             return NotImplemented
     74 other = item_from_zerodim(other)
---> 76 return method(self, other)

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/arraylike.py:19
4, in OpsMixin.__sub__(self, other)
    192 @unpack_zerodim_and_defer("__sub__")
    193 def __sub__(self, other):
--> 194     return self._arith_method(other, operator.sub)
```

```
File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/frame.py:7913,
in DataFrame._arith_method(self, other, op)
   7910 self, other = self._align_for_op(other, axis, flex=True, level=None)
   7912 with np.errstate(all="ignore"):
-> 7913     new_data = self._dispatch_frame_op(other, op, axis=axis)
   7914 return self._construct_result(new_data)

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/frame.py:7956,
in DataFrame._dispatch_frame_op(self, right, func, axis)
   7950     assert self.columns.equals(right.columns)
   7951     # TODO: The previous assertion `assert right._indexed_same(self)
`
   7952     #  fails in cases with empty columns reached via
   7953     #  _frame_arith_method_with_reindex
   7954
   7955     # TODO operate_blockwise expects a manager of the same type
-> 7956     bm = self._mgr.operate_blockwise(
   7957         # error: Argument 1 to "operate_blockwise" of "ArrayManager"
has
   7958         # incompatible type "Union[ArrayManager, BlockManager]"; exp
ected
   7959         # "ArrayManager"
   7960         # error: Argument 1 to "operate_blockwise" of "BlockManager"
has
   7961         # incompatible type "Union[ArrayManager, BlockManager]"; exp
ected
   7962         # "BlockManager"
   7963         right._mgr,  # type: ignore[arg-type]
   7964         array_op,
   7965     )
   7966     return self._constructor_from_mgr(bm, axes=bm.axes)
   7968 elif isinstance(right, Series) and axis == 1:
   7969     # axis=1 means we want to operate row-by-row

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/internals/manag
ers.py:1511, in BlockManager.operate_blockwise(self, other, array_op)
   1507 def operate_blockwise(self, other: BlockManager, array_op) -> BlockM
anager:
   1508     """
   1509     Apply array_op blockwise with another (aligned) BlockManager.
   1510     """
-> 1511     return operate_blockwise(self, other, array_op)

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/internals/ops.p
y:65, in operate_blockwise(left, right, array_op)
     63 res_blks: list[Block] = []
     64 for lvals, rvals, locs, left_ea, right_ea, rblk in _iter_block_pairs
(left, right):
---> 65     res_values = array_op(lvals, rvals)
     66     if (
     67         left_ea
     68         and not right_ea
     69         and hasattr(res_values, "reshape")
     70         and not is_1d_only_ea_dtype(res_values.dtype)
     71     ):
     72         res_values = res_values.reshape(1, -1)
```

```
File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/ops/array_ops.p
y:283, in arithmetic_op(left, right, op)
    279     _bool_arith_check(op, left, right)  # type: ignore[arg-type]
    281     # error: Argument 1 to "_na_arithmetic_op" has incompatible type
    282     # "Union[ExtensionArray, ndarray[Any, Any]]"; expected "ndarray
[Any, Any]"
--> 283     res_values = _na_arithmetic_op(left, right, op)  # type: ignore
[arg-type]
    285 return res_values

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/ops/array_ops.p
y:227, in _na_arithmetic_op(left, right, op, is_cmp)
    219 except TypeError:
    220     if not is_cmp and (
    221         left.dtype == object or getattr(right, "dtype", None) == obj
ect
    222     ):
   (...)
    225         # Don't do this for comparisons, as that will handle complex
numbers
    226         #  incorrectly, see GH#32047
--> 227         result = _masked_arith_op(left, right, op)
    228     else:
    229         raise

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/ops/array_ops.p
y:163, in _masked_arith_op(x, y, op)
    161     # See GH#5284, GH#5035, GH#19448 for historical reference
    162     if mask.any():
--> 163         result[mask] = op(xrav[mask], yrav[mask])
    165 else:
    166     if not is_scalar(y):

TypeError: numpy boolean subtract, the `-` operator, is not supported, use t
he bitwise_xor, the `^` operator, or the logical_xor function instead.
```

The standardization of the dataset may give a `DataConversionWarning`. This informs you that the integer columns in the dataframe are automatically converted to real numbers (`float64`). This is expected and you can therefore ignore this warning. If you want to suppress the warning, you can explicitly convert the integer columns to real numbers

```
# Option 1: Identify all integer columns, remove personal
loan,
# and change their type
intColumns = [c for c in housing_df.columns if
housing_df[c].dtype == 'int']
housing_df[intColumns] =
housing_df[intColumns].astype('float64')
```

Alternatively, you can suppress the warning as follows:

```
# Option 2: use the warnings package to suppress the display
of the warning
import warnings
with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    norm_df = pd.DataFrame(scaler.fit_transform(housing_df),
                           index=housing_df.index,
columns=housing_df.columns)
```

Creating and Using Data Partitions

Table 2.10

Split the dataset into training (60%) and validation (40%) sets. Randomly sample 60% of the dataset into a new data frame `trainData`. The remaining 40% serve as validation.

Partition the dataset into training (50%), validation (30%), and test sets (20%).

In [65]:
```
#VB67
# random_state is set to a defined value to get the same partitions when re-
# training (60\%) and holdout (40\%)
trainData, holdoutData = train_test_split(housing_df, test_size=0.40, random
print('Training  : ', trainData.shape)
print('Holdout   : ', holdoutData.shape)
print()
# training (50\%), validation (30\%), and holdout (20\%)
trainData, temp = train_test_split(housing_df, test_size=0.5, random_state=1
validData, holdoutData = train_test_split(temp, test_size=0.4, random_state=

print('Training  : ', trainData.shape)
print('Validation: ', validData.shape)
print('Holdout   : ', holdoutData.shape)
```

```
Training  :  (3481, 16)
Holdout   :  (2321, 16)

Training  :  (2901, 16)
Validation:  (1740, 16)
Holdout   :  (1161, 16)
```

Building a Predictive Model

Table 2.12

In Python, we use the scikit-learn LinearRegression method to predict house value with the training data, then use the same model to predict values for the validation data. Chapter 6 on linear regression goes into more detail. Table 2.12 shows the training process and the predicted values for the first few records in the training data along with

the actual values and the residuals (prediction errors). Note that the predicted values are often called the fitted values, since they are for the records to which the model was fit.

In [81]:
```
#VB67
# Data loading and preprocessing
from sklearn.linear_model import LinearRegression

# data loading and preprocessing
housing_df = mlba.load_data('WestRoxbury.csv')
housing_df.columns = [s.strip().replace(' ', '_') for s in housing_df.columr
housing_df.REMODEL = housing_df.REMODEL.fillna('None').astype('category')
housing_df = pd.get_dummies(housing_df, prefix_sep='_', drop_first=True)

# create list of predictors and outcome
excludeColumns = ('TOTAL_VALUE', 'TAX')
predictors = [column for column in housing_df.columns if column not in exclu
outcome = 'TOTAL_VALUE'

# partition data
X = housing_df[predictors]
y = housing_df[outcome]
train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.
                                                          random_state=1)
#train
model = LinearRegression()
model.fit(train_X, train_y)

#generate predictions
train_pred = model.predict(train_X)
train_results = pd.DataFrame({
     'TOTAL_VALUE': train_y,
     'predicted': train_pred,
     'residual': train_y - train_pred,
 })
train_results.head()
```

Out[81]:

	TOTAL_VALUE	predicted	residual
2024	392.0	387.726258	4.273742
5140	476.3	430.785540	45.514460
5259	367.4	384.042952	-16.642952
421	350.3	369.005551	-18.705551
1401	348.1	314.725722	33.374278

In [86]:
```
#VB67
#plotting actual and predicted values
plt.plot(train_results.TOTAL_VALUE, train_results.predicted, '.')
plt.xlabel('actual')  # set x-axis label
plt.ylabel('predicted')  # set y-axis label
axes = plt.gca()
#reference line for perfect predictions
```

```
plt.plot(axes.get_xlim(), axes.get_xlim(), '--')
plt.show()
```

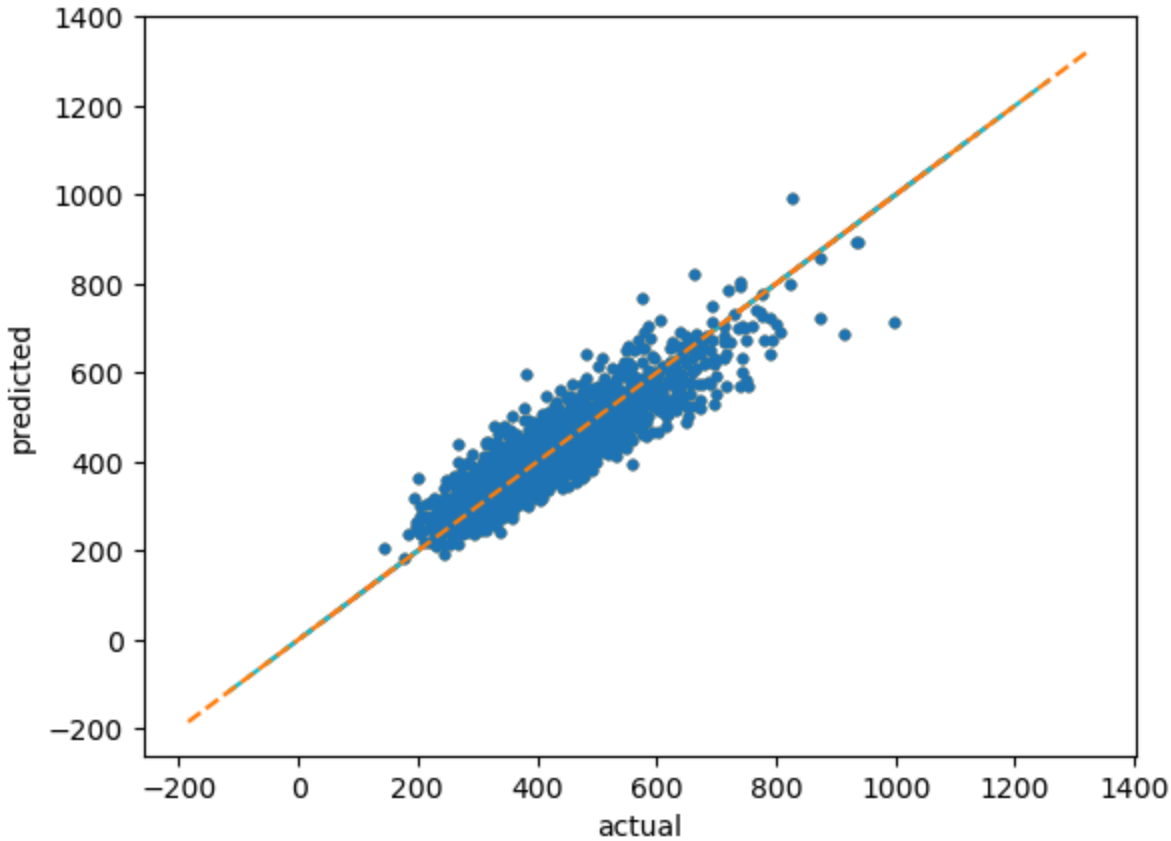

Table 2.13

Prediction for a sample of holdhout data

In [87]:
```
#VB67
# loading and creating subsets from the data set
holdout_pred = model.predict(holdout_X)
holdout_results = pd.DataFrame({
     'TOTAL_VALUE': holdout_y,
     'predicted': holdout_pred,
     'residual': holdout_y - holdout_pred,
 })
holdout_results.head()
```

Out[87]:

	TOTAL_VALUE	predicted	residual
1822	462.0	406.946377	55.053623
1998	370.4	362.888928	7.511072
5126	407.4	390.287208	17.112792
808	316.1	382.470203	-66.370203
4034	393.2	434.334998	-41.134998

Table 2.14

We can use the metrics that scikit-learn provides.

In [88]:
```
#VB67
# scikit-learn calculates the R² (coefficient of determination) regression s
print('Training set r2: ', r2_score(train_results.TOTAL_VALUE, train_results
print('Validation set r2: ', r2_score(holdout_results.TOTAL_VALUE, holdout_r
```

```
Training set r2:  0.8097361461091852
Validation set r2:  0.8171327286147878
```

Or we can use the functions in the mlba package from the textbook.

In [89]:
```
#VB67
# import the utility function regressionSummary
from mlba import regressionSummary

# training set
regressionSummary(y_true=train_results.TOTAL_VALUE, y_pred=train_results.pre

# holdout set
regressionSummary(y_true=holdout_results.TOTAL_VALUE, y_pred=holdout_results
```

```
Regression statistics

Mean Error (ME) : -0.0000
Root Mean Squared Error (RMSE) : 43.0306
Mean Absolute Error (MAE) : 32.6042
Mean Percentage Error (MPE) : -1.1116
Mean Absolute Percentage Error (MAPE) : 8.4886

Regression statistics

Mean Error (ME) : -0.1463
Root Mean Squared Error (RMSE) : 42.7292
Mean Absolute Error (MAE) : 31.9663
Mean Percentage Error (MPE) : -1.0884
Mean Absolute Percentage Error (MAPE) : 8.3283
```

In [ ]: